Pensieve: Harry Potter Chatbot - Project Analysis Report

Project Overview

Pensieve is a retrieval-augmented generation (RAG) chatbot designed to answer questions strictly based on a provided Harry Potter knowledge base. It was developed by **Abdullah Hani Abdellatif Al-Shobaki** and **Abdulah M. M. Abbas** for the **COP4921 – Applied Large Language Models** course at **Bahçeşehir University**.

The project successfully fulfills the requirements of the midterm assignment by implementing a robust, dual-indexing FAISS retrieval system coupled with the Qwen Large Language Model (LLM). It features strict grounding (refusing to answer out-of-domain questions), conversation memory, jailbreak protection, and a themed web interface.

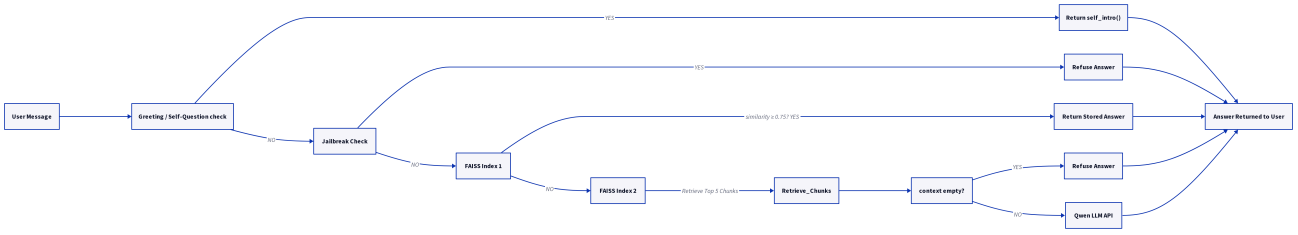
Architecture and Workflow

The chatbot operates using a multi-layered approach to ensure accuracy, security, and efficiency:

1. **Input Validation (Jailbreak Detection):** Before any processing, the user's input is checked against 17 regular expression patterns to detect and block prompt injection or jailbreak attempts (e.g., "forget your instructions", "write python code").
2. **Direct Answer Retrieval (FAISS Index 1):** The system first queries a FAISS index containing only known questions. If the user's query has a high cosine similarity ($\geqslant 0.75$) to a stored question, the system returns the pre-defined answer immediately, bypassing the LLM API call entirely.
3. **Context Retrieval (FAISS Index 2):** If no direct match is found, the system queries a second FAISS index containing the full knowledge base (questions, answers, and raw information). It retrieves the top 5 most relevant text chunks to serve as context.
4. **LLM Generation:** The retrieved context, the conversation history (last 3 turns), and the user's query are combined with a strict system prompt. This prompt instructs the Qwen LLM to answer *only* based on the provided context and to refuse out-of-domain questions.

Flow Diagram

The flow diagram below details the chatbot's decision logic, from initial security checks to dual-index retrieval. It shows how the system efficiently balances instant direct answers with LLM-generated responses to ensure grounded and secure interactions.



File Structure and Analysis

The repository is well-organized into several key components. Below is a detailed breakdown of each file and its purpose:

1. Core Application Files

- **app.py** : This is the entry point for the web application. It uses the Flask framework to serve the user interface and handle API requests. It defines routes for the main page (`/`), the chat endpoint (`/chat`), and a session reset endpoint (`/reset`). It manages conversation history using Flask sessions.
- **chatbot.py** : This file contains the core logic of the chatbot. It handles the entire workflow described in the architecture section. Key responsibilities include:
  - Loading the FAISS indexes and TF-IDF vectorizers.
  - Defining the `SYSTEM_PROMPT` with strict rules for the LLM.
  - Implementing the `JAILBREAK_PATTERNS` regex checks.
  - Providing functions for embedding queries (`embed_query`) and searching the indexes (`search_faiss1` , `search_faiss2`).
  - The main `chat()` function, which orchestrates the flow from input validation to LLM API invocation using the OpenAI-compatible Qwen API.
- **build_index.py** : This is a one-time setup script used to process the raw data and build the retrieval models. It reads the provided Excel file (`harry_potter_data_02.xlsx`), separates Q&A pairs from raw information, and trains two TF-IDF vectorizers (using bigrams). It then creates and saves the two FAISS indexes (`faiss_questions.index` and `faiss_fulldata.index`) and exports the necessary JSON and `.skops` files for the chatbot to use at runtime.

2. Data and Retrieval Assets

- **data/harry_potter_data_02.xlsx** : The source knowledge base provided for the assignment, containing Q&A pairs and raw informational text about Harry Potter.

- **data/questions.json** ** & **** data/answers.json **: Extracted lists of questions and their corresponding answers, used by Index 1 for direct lookup.
- **data/all_texts.json** : A list of all text chunks (Q&A and raw info) used by Index 2 to provide context to the LLM.
- **retrieval/faiss_questions.index** : The FAISS index optimized for exact-match question lookup (Index 1).
- **retrieval/faiss_fulldata.index** : The FAISS index containing the full knowledge base for semantic context retrieval (Index 2).
- **retrieval/vectorizer_questions.skops** ** & **** retrieval/vectorizer_fulldata.skops **: The serialized scikit-learn TF-IDF vectorizers used to convert text into embeddings for FAISS searching.

3. User Interface

- **templates/index.html** : A single-page HTML file containing the chat interface. It features a custom "Harry Potter" theme with dark gold accents, a parchment texture background, and custom CSS animations. It includes JavaScript to handle sending messages to the Flask backend, displaying typing indicators, and managing the chat UI.

4. Configuration and Documentation

- **README.md** : A comprehensive guide to the project, detailing its features, architecture, setup instructions, and how it fulfills the specific requirements of the midterm assignment.
- **requirements.txt** : A list of Python dependencies required to run the project (e.g., Flask, faiss-cpu, scikit-learn, openai).
- **.env.example** : A template file showing the required environment variables (QWEN_API_KEY and FLASK_SECRET_KEY).

Fulfillment of Project Requirements

Based on the provided midterm announcement, the project successfully addresses all requirements:

Requirement	Implementation Details
1 & 2. Grounded Answers Only	Enforced via the SYSTEM_PROMPT (Rules 1 & 6) and by ensuring the LLM only receives context retrieved from the FAISS index. If no context is found, it defaults to "I cannot answer that..".

3. Greetings & Self-Questions	Handled explicitly in `chatbot.py` (`is_greeting` , `is_self_question`) and via `SYSTEM_PROMPT` Rule 2, allowing the bot to introduce itself without revealing internal mechanics.
4. Jailbreak Resilience	Implemented using a robust list of 17 regex patterns (`JAILBREAK_PATTERNS`) that block malicious inputs before they reach the LLM, supplemented by `SYSTEM_PROMPT` Rules 3 & 4.
5. Conversation Memory	The Flask app stores history in the user's session, and `chatbot.py` maintains the last 3 turns (`MAX_HISTORY = 6` messages) to provide context for follow-up questions.
6. Format Manipulation Resistance	Addressed via regex checks (e.g., blocking "in exactly 10 words") and `SYSTEM_PROMPT` Rule 5.
7 & 8. Smart Chunking & Dual FAISS	Fully implemented in `build_index.py` and `chatbot.py` . Index 1 handles direct question matching (skipping the API), while Index 2 retrieves the top 5 chunks for LLM context.
9. Interface	A well-designed, themed web application built with Flask and HTML/CSS/JS.

Technology Stack

- **Backend**: Python, Flask
- **LLM Integration**: Qwen (Alibaba DashScope) via OpenAI-compatible API
- **Vector Database**: FAISS (Facebook AI Similarity Search)
- **Text Vectorization**: scikit-learn (TF-IDF)
- **Frontend**: HTML, CSS, JavaScript
- **Environment Management**: `dotenv`

Reflections on the Project

Difficulties Encountered

Developing a robust RAG chatbot, especially one with strict grounding and security requirements, presents several challenges:

- **Fine-tuning Retrieval Thresholds**: Determining the optimal `SIMILARITY_THRESHOLD` for FAISS Index 1 was crucial. A value too high would lead to unnecessary LLM calls, while a value too low could result in inaccurate direct answers. This required iterative testing and adjustment.
- **Comprehensive Jailbreak Protection**: Crafting a comprehensive set of regex patterns for jailbreak and injection detection was complex. It involved anticipating various adversarial prompts and ensuring that legitimate queries were not inadvertently blocked. Balancing strictness with usability was a delicate act.
- **Prompt Engineering for Grounding**: Designing the `SYSTEM_PROMPT` to effectively enforce grounded answers and prevent hallucinations, while still allowing for natural conversation (e.g., greetings), required careful iteration. The rules needed to be clear and unambiguous for the LLM.

Enjoyable Aspects

Despite the challenges, several aspects of the project were particularly rewarding:

- **Seeing the Dual FAISS System Work**: Implementing and observing the efficiency of the dual FAISS indexing system—where simple questions were answered instantly without an LLM call—was very satisfying. It highlighted the power of hybrid retrieval approaches.
- **Creating a Themed UI**: Designing the Harry Potter-themed web interface with custom CSS and JavaScript was an enjoyable creative process. The visual appeal significantly enhanced the user experience and made the project feel more polished.
- **Integrating LLM Capabilities**: Successfully integrating the Qwen LLM and seeing it generate coherent, context-aware responses (within the defined constraints) was a testament to the power of modern language models and effective RAG implementation.
- **Problem-Solving Security Challenges**: Tackling the jailbreak and injection protection aspects was an engaging problem-solving exercise, reinforcing the importance of security in AI applications.

Conclusion

The Pensieve project is a well-structured and complete implementation of a RAG-based chatbot. It demonstrates a clear understanding of vector databases (FAISS), prompt engineering, and security considerations (jailbreak prevention) when working with Large Language Models. The code is clean, well-commented, and effectively separates concerns between data processing, core logic, and the web interface.